

正则表达式 (Python版)

语法

普通字符

[ABC] — 匹配[]中的所有字符

[^ABC] — 匹配除了[]中的所有字符

[A-Z] — 匹配区间内所有字符

[a-z]

[0-9]

非打印字符

.

[\s\S] — \s匹配所有空白符(include \n), \S匹配所有非空白符

[w] == [A-Za-z0-9_] , 匹配字母\数字\下划线

[d] == [0-9], digit

定位符

^ — 匹配字符串开头(except in [])

\$ — 匹配字符串结尾

\b — 匹配单词边界

\B — 匹配非单词边界

限定符

*

+

?

{

.

\

|

特殊字符

\cx — 控制字符. x in [A-Za-z], 表Ctrl+x. 如\cM表 Ctrl+M或者回车符

\f — 换页符. == \x0c 和 \cL

More important

\n — 换行符. == \x0a 和 \cJ

\r — 回车符. == \x0d 和 \cM

\s — 任意空白字符. == [\f\n\r\t\v]

\S — 任意非空白字符

\t — 制表符. == \x09 和 \cI (大写I)

\v — 垂直制表符. == \x0b 和 \cK

换行: 移至下一行
回车: 移至行开头

大小写取反

匹配前面的表达式 0..n 次

1..n

0..1

{n} 精确匹配n次.

{n,} 至少n次

{n,m}表示最少n次, 最多m次.

1 (except \n)

转义字符, 标记下一个字符为: 特殊字符\原义字符
\向后引用\进制转义符

分组

逻辑或

贪婪

*与+都是贪婪的, 即会尽可能多地匹配字符

添加 ? 实现非贪婪

() 有两个功能

- 1. 分组: 把几个字符看成一个整体 (比如 (abc)+)。
- 2. 捕获 (缓存): 引擎会把括号里匹配到的内容存进内存, 编号为 Group 1, Group 2... 供你后面使用。

?:

普通捕获 (abc): 匹配 abc , 并存为 Group 1。

非捕获 (?:abc): 匹配 abc , 但不存, 不给编号。

选择

断言

符号	名称	逻辑含义
(?=exp)	正向肯定预查	右边必须是 exp
(?<exp)	反向肯定预查	左边必须是 exp
(?!exp)	正向否定预查	右边不能是 exp
(?<exp)	反向否定预查	左边不能是 exp

test code

```
import re
text = "Price: $100, €200, 300元, 400USD"
res1 = re.findall(r"\d+(?=元)", text)
res2 = re.findall(r"(?<=€)\d+", text)
res3 = re.findall(r"\d+(?![A-Z0-9])", text)
res4 = re.findall(r"(?<![€0-9])\d+", text)]
```

- 1. 右边是'元'的数字: ['300']
- 2. 左边是'€'的数字: ['100']
- 4. 右边没有大写字母的数字: ['100', '200', '300']
- 4. 左边没有货币符号的数字: ['300', '400']

反向引用

```
str = "Is is the cost of of gasoline going up up?"
pattern = r"\b([a-zA-Z]+) \1\b"
matches = [m.group(0) for m in re.finditer(pattern, str, flags=re.IGNORECASE)]
print(matches) # 输出: ['Is is', 'of of', 'up up']
```

()匹配到后, \1 指向group[1], 即后面跟前面捕获存储的匹配项一致

```
url = "https://www.runoob.com:80/html/html-tutorial.html"
pattern = r"(\w+):\/\/([^\:\/]+)(:\d+)?(?:[^\s]*)"
match_obj = re.search(pattern, url)

if match_obj:
    # 获取完整匹配内容
    full_match = match_obj.group(0)
    # 获取各个分组内容
    sub_groups = match_obj.groups()
    arr = [full_match] + list(sub_groups)
    for item in arr:
        print(item)
```

https
www.runoob.com
:80
/html/html-tutorial.html

- 1. (\w+): 协议部分。匹配字母数字, 直到遇到 :。这里抓到了 https。
- 2. :\/\/: 固定格式。匹配 ://, 注意斜杠在正则里通常需要转义。
- 3. ([^\:\/]+): 域名部分。[^\:\/] 表示"除了斜杠和冒号以外的任何字符"。它会一直跑, 直到撞上冒号 (端口) 或斜杠 (路径)。这里抓到了 www.runoob.com。
- 4. (:d+)? : 端口部分。? 表示这一块是可选的。匹配冒号后面跟着的数字。这里抓到了 :80。
- 5. ([^\s]*) : 路径部分。匹配剩下的一切, 直到遇到 # (锚点) 或空格。这里抓到了 /html/html-tutorial.html。

修饰符

概览

全局匹配

re.search 只匹配第一个

re.findall 匹配所有

re.I / re.IGNORECASE,忽略大小写

re.M / re.MULTILINE,多行模式。改变 ^ 和 \$ 的行为, 使其匹配每行的开头和结尾。

re.S / re.DOTALL,点通配模式

re.X / re.VERBOSE,扩展模式。忽略正则表达式中的空白和注释, 使复杂的正则更容易阅读。

re.A / re.ASCII,"ASCII"模式。使 \w, \W, \b, \B, \d, \D 等仅匹配 ASCII 字符 (Python 3 默认支持 Unicode)。

扩展

组合使用

使用按位或

内联: 直接将修饰符写进则这不都是最前面

```
text = "runoobgoogle\ntaobao\nrunoobweibo"
n1 = re.findall(r"^runoob", text) # ['runoob']
n2 = re.findall(r"^runoob", text, flags=re.M) # ['runoob', 'runoob']
```

```
text = "google\nrunoob\ntaobao"
n1 = re.search(r"google.", text) # None
n2 = re.search(r"google.", text, flags=re.S) # google\n
```

e.g

密码强度正则表达式, 最少6位, 包括至少1个大写字母, 1个小写字母, 1个数字, 1个特殊字符

^(?=.*{6,})(?=.*\d)(?=.*[A-Z])(?=.*[a-z])(?=.*!@#%&'&*?)).*\$

1. 头尾的框架

- ^ 和 \$: 这俩你之前应该学过了, 分别代表字符串的开头和结尾。
- .* (出现了两次): 代表匹配任意字符零次或多次。

2. 核心的断言组

中间那一长串括号, 就是五个并排站着的门卫, 它们在同一位置同时往前看, 必须全部点头同意, 正则才会继续往下走:

- (?=.*{6,}): 门卫 1 往前看, 要求前面的字符串长度至少为 6 个字符及以上。
- (?=.*\d): 门卫 2 往前看, 要求在任意字符 (.*) 之后, 必须出现至少一个数字 (\d)。
- (?=.*[A-Z]): 门卫 3 往前看, 要求必须包含至少一个大写字母。
- (?=.*[a-z]): 门卫 4 往前看, 要求必须包含至少一个小写字母。
- (?=.*[!@#%&'&*?]): 门卫 5 往前看, 要求必须包含至少一个括号里的特殊字符或空格。